Day 14: Reading Files

Real security work means reading data from **files**: log files, configuration files, lists of hostnames, exported reports. Today you learn to read them safely. (You touched on open() and json.load() on Day 13; now you'll see how it works.)

1. Opening Files in Read Mode

open() connects your program to a file.

python

```
f = open("sample.txt", "r")
```

- The first argument is the **file name or path**.
- The second is the **mode**. "r" means **read**, and it is the default, so open("sample.txt") does the same.
- The result is a **file object** (f) that you use to read the contents.

Common modes:

Mode	Meaning	Notes
"r"	Read	File must exist, otherwise an error
"w"	Write	Creates the file, **erases** existing content
"a"	Append	Adds to the end, keeps existing content
"r+"	Read and write	File must exist

Today uses only "r". Writing comes tomorrow.

Always close a file when finished:

python

```
f = open("sample.txt", "r")
content = f.read()
f.close()
```

Forgetting close() can leak resources or lock the file. Section 4 shows how to avoid the problem entirely.

Where is the file?

Python looks for relative paths in the **current working directory**, which is the folder you ran the command from, not necessarily where your script lives.

python

```
open("sample.txt")          # same folder as where you run Python
open("logs/auth.log")       # inside a subfolder
```

```
open("C:\\Users\\lab\\sample.txt") # full Windows path (note the doubled backslashes)

open(r"C:\Users\lab\sample.txt")   # raw string, easier on Windows

open("/home/lab/sample.txt")       # full Linux/macOS path
```

Tip: to find where you are, use:

```
python

import os

print(os.getcwd())
```

Creating a practice file

Create sample.txt in your course folder with this content (use VS Code, one item per line):

```
alice login ok

bob login failed

carol login ok

bob login failed

dave login failed
```

What if the file is missing?

```
python

open("missing.txt")

# FileNotFoundError: [Errno 2] No such file or directory: 'missing.txt'
```

Check first, or catch the error:

```
python

import os


if os.path.exists("sample.txt"):

    print("File found")

else:

    print("File not found")

python

try:

    f = open("missing.txt")

except FileNotFoundError:

    print("That file does not exist.")
```

(You saw try/except on Day 13. A full lesson on error handling is coming.)

2. Using read() and readlines()

read(): the whole file as one string

python

```
f = open("sample.txt", "r")
content = f.read()
f.close()

print(content)
print(type(content))     # <class 'str'>
print(len(content))      # number of characters
```

Output:

```
alice login ok
bob login failed
carol login ok
bob login failed
dave login failed
<class 'str'>
98
```

(The exact character count depends on your line endings.)

Newlines are part of the text. Each line ends with \n, which is why print(repr(content)) shows it:

python

```
print(repr(content))
# 'alice login ok\nbob login failed\n...'
```

Reading only part of the file:

python

```
f = open("sample.txt")
print(f.read(5))     # first 5 characters: alice
print(f.read(6))     # the NEXT 6 characters
f.close()
```

The file object remembers its position, like a bookmark. Once everything has been read, further read() calls return an empty string "".

python

```
f = open("sample.txt")
first = f.read()
second = f.read()
print(len(first), len(second))     # 98 0
f.close()
```

Working with the string you got:

python

```
failed = content.count("failed")
print(f"'failed' appears {failed} times")      # 3


lines = content.splitlines()                  # list of lines WITHOUT \n
print(lines)
```

readlines(): a list of lines

python

```
f = open("sample.txt")
lines = f.readlines()
f.close()


print(lines)
# ['alice login ok\n', 'bob login failed\n', 'carol login ok\n', ...]
print(len(lines))        # 5
print(lines[0])          # alice login ok (plus a blank line, because of \n)
```

Each item is a string that **keeps its trailing \n.** Remove it with strip():

python

```
clean = [line.strip() for line in lines]
print(clean)
# ['alice login ok', 'bob login failed', 'carol login ok', ...]
```

readline(): one line at a time

python

```
f = open("sample.txt")
```

```
print(f.readline().strip())     # alice login ok
print(f.readline().strip())     # bob login failed
f.close()
```

Which should you use?

Method	Returns	Best for
read()	One big string	Small files, searching for text, counting
readlines()	List of lines	Small files when you need the lines in a list
readline()	One line	Reading a header or the first line
Looping over f	One line per pass	**Large files** (Section 3)

Memory warning: read() and readlines() load the **entire** file into memory. A multi-gigabyte log would use that much RAM or crash the program. For big files, loop line by line.

3. Reading Files Line by Line

The file object is **iterable,** so a for loop reads one line per pass, without loading everything at once.

python

```
f = open("sample.txt")

for line in f:
    print(line.strip())

f.close()
```

Output:

```
alice login ok
bob login failed
carol login ok
bob login failed
dave login failed
```

Why strip()? Without it each print adds its own newline on top of the one already in line, so you'd see blank lines between entries.

python

```
for line in f:
    print(line)          # double spacing
```

```
    print(line, end="")      # also fine: removes print's newline
    print(line.strip())      # most common
```

Numbering the lines with enumerate() (Day 10):

```
python
for number, line in enumerate(f, start=1):
    print(f"{number}: {line.strip()}")
```

Filtering lines (the core of log analysis):

```
python
for line in f:
    line = line.strip()
    if "failed" in line:
        print(line)
```

Skipping blank lines (Day 8's continue):

```
python
for line in f:
    line = line.strip()
    if line == "":
        continue
    print(line)
```

Splitting each line into fields:

```
python
for line in f:
    user, _, result = line.split()
    print(f"{user} -> {result}")
```

This works only if every line has exactly three words. A malformed line causes a ValueError, so real parsers check first:

```
python
parts = line.split()
if len(parts) != 3:
    continue                # ignore badly formed lines
user, _, result = parts
```

Counting with a dictionary (Day 11):

python

```
failures = {}


for line in f:
    parts = line.split()
    if len(parts) == 3 and parts[2] == "failed":
        user = parts[0]
        failures[user] = failures.get(user, 0) + 1


print(failures)       # {'bob': 2, 'dave': 1}
```

Reading just the first few lines:

python

```
for i, line in enumerate(f):
    if i >= 3:
        break
    print(line.strip())
```

Reading a file only once: after a loop finishes, the "bookmark" is at the end. To read again, reopen the file or use f.seek(0) to jump back to the start.

4. Using with open()

The safest way to work with files is the **with statement** (you used it on Day 13).

python

```
with open("sample.txt", "r") as f:
    for line in f:
        print(line.strip())


print("File is now closed")
```

Why it's better:

- The file is **closed automatically** when the indented block ends.
- It closes even if an **error** happens inside the block.
- You can't forget close().

The file is only open inside the block. Do all your reading there:

python

```
with open("sample.txt") as f:
    pass


f.read()
# ValueError: I/O operation on closed file
```

Pattern 1: read everything, then work outside the block

python

```
with open("sample.txt") as f:
    lines = f.read().splitlines()


# file is closed; work with the list
print(len(lines))
```

Pattern 2: process line by line inside the block

python

```
failed_count = 0


with open("sample.txt") as f:
    for line in f:
        if "failed" in line:
            failed_count += 1


print(failed_count)
```

Handling a missing file safely:

python

```
try:
    with open("sample.txt") as f:
        content = f.read()
except FileNotFoundError:
    print("Error: sample.txt was not found.")
else:
```

```
print(f"Read {len(content)} characters")
```

Specifying the encoding. Text files are stored as bytes using an *encoding*. Spell it out to avoid surprises on different systems:

python

```
with open("sample.txt", "r", encoding="utf-8") as f:
    text = f.read()
```

Without it, Python uses the system default, which on some Windows machines is not UTF-8. Real logs sometimes contain odd bytes; errors="replace" stops a crash on them:

python

```
open("auth.log", encoding="utf-8", errors="replace")
```

Opening two files at once:

python

```
with open("a.txt") as f1, open("b.txt") as f2:
    print(f1.readline().strip(), f2.readline().strip())
```

Security note: when a program takes a **file name from user input**, never trust it blindly. A user could enter a path to a sensitive file. In your own lab programs, restrict reading to a specific folder or an approved list of names.

Putting It All Together: Log File Analyzer

This reads a **log file you create yourself**, with made-up data. It never touches real system logs.

First create auth_sample.log:

```
10:01 alice login ok 10.0.0.7
10:02 bob login failed 10.0.0.5


10:03 bob login failed 10.0.0.5
10:04 carol login ok 10.0.0.9
10:05 bob login failed 10.0.0.5
this line is damaged
10:06 dave login failed 10.0.0.9
10:07 bob login ok 10.0.0.5
```

Create day14_program.py:

python

```
# Program: Day 14 reading files
```

```
# Purpose: Analyze a SAMPLE log file that I created myself (practice only)


LOG_FILE = "auth_sample.log"

ALERT_THRESHOLD = 3



def read_log_lines(filename):

    """Return a list of non-empty, stripped lines, or None if the file is missing."""

    try:

        with open(filename, "r", encoding="utf-8") as f:

            return [line.strip() for line in f if line.strip() != ""]

    except FileNotFoundError:

        return None



def parse_line(line):

    """Return (time, user, result, ip), or None if the line is malformed."""

    parts = line.split()

    if len(parts) != 5 or parts[2] != "login":

        return None

    time, user, _, result, ip = parts

    return time, user, result, ip



def analyze(lines):

    """Count failures per user and per IP, and count bad lines."""

    by_user = {}

    ips = set()

    ok_count = 0

    bad_lines = 0
```

```
    for line in lines:
        record = parse_line(line)
        if record is None:
            bad_lines += 1
            continue

        time, user, result, ip = record
        if result == "failed":
            by_user[user] = by_user.get(user, 0) + 1
            ips.add(ip)
        elif result == "ok":
            ok_count += 1

    return by_user, ips, ok_count, bad_lines



def print_report(total, by_user, ips, ok_count, bad_lines):
    print("=" * 36)
    print("  LOG FILE ANALYZER")
    print("=" * 36)
    print(f"Lines read:          {total}")
    print(f"Successful logins:   {ok_count}")
    print(f"Failed logins:       {sum(by_user.values())}")
    print(f"Malformed lines:     {bad_lines}")
    print(f"Unique failing IPs:  {len(ips)}")
    print()

    print("Failures per user:")
    if not by_user:
        print("  none")
    for user, count in sorted(by_user.items(), key=lambda item: item[1], reverse=True):
```

```
        flag = "  <-- ALERT" if count >= ALERT_THRESHOLD else ""
        print(f"  {user:<8}{count:>3}{flag}")


def main():
    lines = read_log_lines(LOG_FILE)
    if lines is None:
        print(f"Error: '{LOG_FILE}' not found. Create it in the folder you run Python from.")
        return

    by_user, ips, ok_count, bad_lines = analyze(lines)
    print_report(len(lines), by_user, ips, ok_count, bad_lines)


if __name__ == "__main__":
    main()
```

Output:

```
====================================
  LOG FILE ANALYZER
====================================
Lines read:          9
Successful logins:   3
Failed logins:       4
Malformed lines:     1
Unique failing IPs:  2

Failures per user:
  bob        3  <-- ALERT
  dave       1
```

Day 15: Writing Files

Yesterday you read files. Today you create and extend them. Security tools constantly produce output: scan reports, alert logs, audit summaries, exported findings. You touched on writing on Day 13 with json.dump(); now you will see how plain text writing works.

1. Write Mode and Append Mode

Write mode: "w"

python

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("First note")
```

- If notes.txt **does not exist**, Python creates it.
- If it **does exist**, Python **erases all its content** and starts fresh. There is no warning and no undo.

Append mode: "a"

python

```
with open("notes.txt", "a", encoding="utf-8") as f:
    f.write("Another note")
```

- Creates the file if missing.
- If it exists, the old content is **kept** and new text is added at the **end**.

Comparison

Mode	File missing	File exists	Typical use
"r"	Error	Reads it	Reading
"w"	Creates it	**Erases it**, then writes	Fresh reports, regenerated output
"a"	Creates it	Adds to the end	Logs that grow over time
"x"	Creates it	**Error** (refuses to overwrite)	Safe creation
"r+"	Error	Read and write	Editing in place (advanced)

Demonstration:

python

```
with open("demo.txt", "w") as f:
    f.write("one\n")


with open("demo.txt", "a") as f:
    f.write("two\n")
```

```
with open("demo.txt", "w") as f:      # erases "one" and "two"!
    f.write("three\n")


with open("demo.txt") as f:
    print(f.read())                   # three
```

Protecting existing files with "x" and os.path.exists

Exclusive mode refuses to overwrite:

```
python

try:
    with open("report.txt", "x") as f:
        f.write("New report\n")
except FileExistsError:
    print("report.txt already exists, not overwriting.")
```

Or ask first:

```
python

import os


if os.path.exists("report.txt"):
    answer = input("report.txt exists. Overwrite? (yes/no): ").strip().lower()
    if answer != "yes":
        print("Cancelled.")
        raise SystemExit
```

(raise SystemExit ends the program. In a function you would simply return.)

Choosing the right mode

- Building a **new report each run**: "w".
- Keeping a **running history** (an activity log): "a".
- Not sure and the data matters: "x", or ask first.

Safety habit: never open a file in "w" mode if you only wanted to look at it, and double-check the file name before writing. Opening your own script or an important file with "w" destroys it instantly.

2. Using write()

write() puts a **string** into the file.

python

```
with open("report.txt", "w", encoding="utf-8") as f:
    f.write("Scan report")
    f.write("Target: Lab-Server-01")
```

Result in the file:

Scan reportTarget: Lab-Server-01

Everything ran together, because write() adds **exactly what you give it**. No automatic newline (unlike print()). That leads to the next section.

write() accepts only strings.

python

```
with open("data.txt", "w") as f:
    f.write(80)          # TypeError: write() argument must be str, not int
    f.write(str(80))     # correct
    f.write(f"Port {80}")    # also correct
```

write() returns the number of characters written (rarely needed):

python

```
n = f.write("hello")       # n is 5
```

Writing with f-strings:

python

```
host = "192.168.56.101"
port = 22
state = "open"

with open("result.txt", "w", encoding="utf-8") as f:
    f.write(f"{host}:{port} is {state}\n")
```

Writing several lines from a list: writelines()

python

```
lines = ["alpha\n", "beta\n", "gamma\n"]

with open("names.txt", "w", encoding="utf-8") as f:
    f.writelines(lines)
```

writelines() also adds **no newlines;** they must already be in the strings.

Writing with print(): you can redirect print() into a file, and it adds the newline for you.

python

```
with open("report.txt", "w", encoding="utf-8") as f:
    print("Scan report", file=f)
    print("Target: Lab-Server-01", file=f)
```

This is very convenient for reports, since you can reuse print formatting, sep, and end.

Data is buffered. Python may hold text in memory briefly before writing it to disk. Leaving the with block (or calling f.close()) makes sure everything is saved. If your program crashes in the middle without with, recent writes can be lost.

3. Adding New Lines

The newline character is \n. Add it yourself.

python

```
with open("report.txt", "w", encoding="utf-8") as f:
    f.write("Scan report\n")
    f.write("Target: Lab-Server-01\n")
    f.write("Status: Safe\n")
```

File content:

```
Scan report
Target: Lab-Server-01
Status: Safe
```

Blank lines use \n\n or a separate "\n":

python

```
f.write("Section 1\n\n")       # text, then an empty line
f.write("\n")                  # an empty line on its own
```

Writing a list, one item per line:

python

```
ports = [22, 80, 443]

with open("ports.txt", "w", encoding="utf-8") as f:
    for port in ports:
        f.write(f"{port}\n")
```

Or in one step with join():

python

```
with open("ports.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(str(p) for p in ports) + "\n")
```

Appending without gluing lines together. A classic bug: the last line of an existing file has no trailing newline, so appended text sticks to it. The fix is to **end every line you write with \n**, not start it with one.

python

```
# Good habit: every record ends with a newline
with open("activity.log", "a", encoding="utf-8") as f:
    f.write("10:01 started\n")
    f.write("10:02 stopped\n")
```

Newlines on different systems: Windows uses \r\n internally, but Python in text mode translates \n automatically. Always write \n; never write \r\n yourself.

Adding timestamps to log entries:

python

```
from datetime import datetime


def log_event(message, filename="activity.log"):
    stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(filename, "a", encoding="utf-8") as f:
        f.write(f"{stamp} {message}\n")


log_event("Program started")
log_event("Report generated")
```

File content:

2026-10-03 10:15:42 Program started

2026-10-03 10:15:42 Report generated

(Your timestamps will differ.) Timestamps are essential in security logs, because investigators need to know **when** things happened.

Never write secrets into logs. Don't log passwords, tokens, or full personal data. Log the *event* ("login failed for user bob"), not the secret.

4. Creating Text Reports

A good report has a header, a body, and a summary. Build it in a function that **returns the text**, then save it. That way you can both print it and write it to a file.

Step 1: build the text

python

```
def build_report(title, rows):
    lines = []
    lines.append("=" * 40)
    lines.append(f"  {title}")
    lines.append("=" * 40)

    for name, value in rows:
        lines.append(f"{name:<20}{value}")

    lines.append("-" * 40)
    lines.append(f"Total items: {len(rows)}")
    return "\n".join(lines) + "\n"
```

Step 2: save it

python

```
def save_report(text, filename):
    with open(filename, "w", encoding="utf-8") as f:
        f.write(text)
```

Step 3: use both

python

```
rows = [("Lab-Server-01", "online"), ("Kali-VM", "online"), ("Old-Router", "offline")]

report = build_report("DEVICE STATUS", rows)
print(report)                    # show on screen
save_report(report, "status.txt")      # save to disk
```

Why separate building from saving? One function does one job (Day 12). You can later send the same text to a screen, a file, or an email without rewriting it.

Timestamped file names

Avoid overwriting by putting the date in the name:

python

```
from datetime import datetime


stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
filename = f"report_{stamp}.txt"       # report_20261003_101542.txt
```

Keeping reports tidy

- Save into a dedicated folder, creating it if needed:

python

```
import os
os.makedirs("reports", exist_ok=True)
path = os.path.join("reports", filename)
```

- Use os.path.join() instead of typing slashes, so the path works on Windows, Linux, and macOS.

Safety notes for reports

- Reports from real security work are **sensitive**: they describe weaknesses. Store them carefully, and don't commit them to public GitHub.
- Only generate reports about your **own lab systems or data you are authorized to analyze**.
- Don't overwrite your original evidence: write analysis results to a **new** file and leave the source log untouched.

Putting It All Together: Log Analysis Report Writer

This extends the Day 14 analyzer. It **reads** your sample log and **writes** a report file plus an activity log. It uses only the made-up auth_sample.log from yesterday. Nothing touches real system logs.

Create day15_program.py:

python

```
# Program: Day 15 writing files
# Purpose: Read a SAMPLE log, write a text report and an activity log (practice only)


import os
from datetime import datetime


LOG_FILE = "auth_sample.log"
REPORT_DIR = "reports"
ACTIVITY_LOG = "activity.log"
```

```
ALERT_THRESHOLD = 3



def log_event(message):
    """Append a timestamped line to the activity log."""
    stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(ACTIVITY_LOG, "a", encoding="utf-8") as f:
        f.write(f"{stamp} {message}\n")



def read_log_lines(filename):
    """Return stripped, non-empty lines, or None if the file is missing."""
    try:
        with open(filename, "r", encoding="utf-8") as f:
            return [line.strip() for line in f if line.strip() != ""]
    except FileNotFoundError:
        return None



def parse_line(line):
    """Return (time, user, result, ip), or None if malformed."""
    parts = line.split()
    if len(parts) != 5 or parts[2] != "login":
        return None
    time, user, _, result, ip = parts
    return time, user, result, ip



def analyze(lines):
    by_user = {}
    ips = set()
```

```
    ok_count = 0
    bad_lines = 0

    for line in lines:
        record = parse_line(line)
        if record is None:
            bad_lines += 1
            continue
        _, user, result, ip = record
        if result == "failed":
            by_user[user] = by_user.get(user, 0) + 1
            ips.add(ip)
        elif result == "ok":
            ok_count += 1

    return by_user, ips, ok_count, bad_lines


def build_report(total, by_user, ips, ok_count, bad_lines):
    """Return the report as one string."""
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    lines = []
    lines.append("=" * 38)
    lines.append("  LOGIN ANALYSIS REPORT")
    lines.append("=" * 38)
    lines.append(f"Generated:          {now}")
    lines.append(f"Source file:        {LOG_FILE}")
    lines.append("")
    lines.append(f"Lines read:         {total}")
    lines.append(f"Successful logins:  {ok_count}")
    lines.append(f"Failed logins:      {sum(by_user.values())}")
```

```
    lines.append(f"Malformed lines:     {bad_lines}")
    lines.append(f"Unique failing IPs:  {len(ips)}")
    lines.append("")
    lines.append("Failures per user:")

    if not by_user:
        lines.append("  none")
    for user, count in sorted(by_user.items(), key=lambda item: item[1], reverse=True):
        flag = "  <-- ALERT" if count >= ALERT_THRESHOLD else ""
        lines.append(f"  {user:<8}{count:>3}{flag}")

    lines.append("")
    lines.append("-" * 38)
    lines.append("End of report")
    return "\n".join(lines) + "\n"


def save_report(text):
    """Save the report with a timestamped name and return the path."""
    os.makedirs(REPORT_DIR, exist_ok=True)
    stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    path = os.path.join(REPORT_DIR, f"report_{stamp}.txt")
    with open(path, "x", encoding="utf-8") as f:     # "x": never overwrite
        f.write(text)
    return path


def main():
    log_event("Program started")

    lines = read_log_lines(LOG_FILE)
```

```
    if lines is None:
        print(f"Error: '{LOG_FILE}' not found.")
        log_event(f"ERROR: {LOG_FILE} not found")
        return

    by_user, ips, ok_count, bad_lines = analyze(lines)
    report = build_report(len(lines), by_user, ips, ok_count, bad_lines)

    print(report)
    path = save_report(report)
    print(f"Report saved to: {path}")

    log_event(f"Report saved to {path}")
    log_event("Program finished")


if __name__ == "__main__":
    main()
```

Output on screen:

```
========================================
  LOGIN ANALYSIS REPORT
========================================
Generated:           2026-10-03 10:15:42
Source file:         auth_sample.log

Lines read:          9
Successful logins:   3
Failed logins:       4
Malformed lines:     1
Unique failing IPs:  2
```

Failures per user:

bob 3 <-- ALERT

dave 1

End of report

Practice Tasks

1. Write a program that creates hello.txt containing three lines of your choice, then read it back and print it.
2. Run a program that appends one line to visits.txt each time. Run it three times and check the file.
3. Prove how "w" behaves: write “first”, then write “second” in "w" mode, and read the file.
4. Write the numbers 1 to 10, one per line, to numbers.txt.
5. Write the multiplication table of a number the user enters to table.txt.